



QB 02: Database Management Systems (DBMS) — Question Bank with Answer Keys

Course: Database Management Systems (Theory + Lab)

Target: VTU MCA Semester 1

Scheme: VTU 2022 / 2024 Scheme (IPCC)

Contents: ER Modeling, Relational Algebra, SQL Practice, Normalization Numericals, ACID & Concurrency, 15 MCQs with Explanations



Syllabus Outline (VTU 5 Modules)

- **Module 1:** Database System Concepts, 3-Tier Architecture, Data Independence, Entity-Relationship (ER) & Extended ER (EER) Modeling.
- **Module 2:** Relational Data Model, Constraints, Relational Algebra Operations (Selection, Projection, Join, Union, Difference, Cartesian Product), Tuple Relational Calculus.
- **Module 3:** SQL Programming (DDL, DML, DCL, TCL), Nested Subqueries, Correlated Queries, Joins, Aggregations (`GROUP BY` , `HAVING`), Views, Stored Procedures, and Triggers.
- **Module 4:** Relational Database Design, Functional Dependencies, Normal Forms (1NF, 2NF, 3NF, BCNF), Lossless Join Decomposition, Dependency Preservation.
- **Module 5:** Transaction Management, ACID Properties, Serializability (Conflict & View), Concurrency Control (2PL, Strict 2PL, Timestamp Ordering), Deadlocks, and Crash Recovery (WAL, Checkpoints).



SECTION 1: High-Yield MCQs with Answer Key

Q1. Which level of database architecture defines how data is physically stored on disk?

- A) External level
- B) Conceptual level
- C) Internal / Physical level
- D) View level

Answer: C

Explanation: The internal level (physical schema) describes physical storage structures, access paths, indexing, and compression methods.

Q2. In an ER diagram, how is a "Weak Entity Set" represented?

- A) Dotted rectangle
- B) Double rectangle
- C) Diamond
- D) Ellipse

Answer: B

Explanation: A weak entity (an entity that cannot be uniquely identified by its own attributes alone) is shown in a double rectangle; its identifying relationship is a double diamond.

Q3. Which relational algebra operator selects rows that satisfy a given predicate?

- A) Projection (π)
- B) Selection (σ)
- C) Join ($\bowtie$)
- D) Division ($\div$)

Answer: B

Explanation: The sigma (σ) operator filters rows (tuples) based on a condition; pi (π) projects columns (attributes).

Q4. What does the `HAVING` clause do in an SQL query?

- A) Filters rows before grouping
- B) Filters groups produced by the `GROUP BY` clause based on aggregate conditions
- C) Creates a temporary index
- D) Replaces the `WHERE` clause completely

Answer: B

Explanation: `WHERE` filters individual rows before aggregation; `HAVING` filters aggregated groups after `GROUP BY`.

Q5. A relation R is in 3NF if for every non-trivial functional dependency $X \rightarrow Y$, which of the following holds?

- A) X is a superkey OR Y is a prime attribute
- B) X is a candidate key AND Y is a prime attribute
- C) Y is a superkey
- D) No multivalued dependencies exist

Answer: A

Explanation: For 3NF, in every $X \rightarrow Y$, either X is a superkey or Y is part of some candidate key (prime attribute). (If only X is a superkey, that is BCNF).

Q6. Which ACID property ensures that all operations in a transaction are completed or none are?

- A) Consistency
- B) Isolation
- C) Durability
- D) Atomicity

Answer: D

Explanation: Atomicity ("all-or-nothing") guarantees that if any part of a transaction fails, the entire transaction is aborted and rolled back.

Q7. In Strict Two-Phase Locking (Strict 2PL), when are exclusive (write) locks released?

- A) As soon as the write operation is complete
- B) During the shrinking phase
- C) Only after the transaction commits or aborts
- D) Immediately before the next lock is requested

Answer: C

Explanation: Strict 2PL holds all exclusive (X) locks until the transaction terminates (commits/aborts) to prevent cascading rollbacks.

Q8. What SQL constraint prevents duplicate values while permitting NULL values?

- A) PRIMARY KEY
- B) UNIQUE
- C) CHECK
- D) FOREIGN KEY

Answer: B

Explanation: PRIMARY KEY forbids both duplicates and NULL s. UNIQUE enforces uniqueness across non-null values while permitting NULL entries (depending on RDBMS standards).

Q9. What is a "phantom read" anomaly in database isolation?

- A) A transaction reads data that was never committed
- B) A transaction reads the same row twice and finds different column values
- C) A transaction re-executes a query reading a set of rows satisfying a condition and finds additional rows inserted by another committed transaction
- D) A transaction overwrites uncommitted data

Answer: C

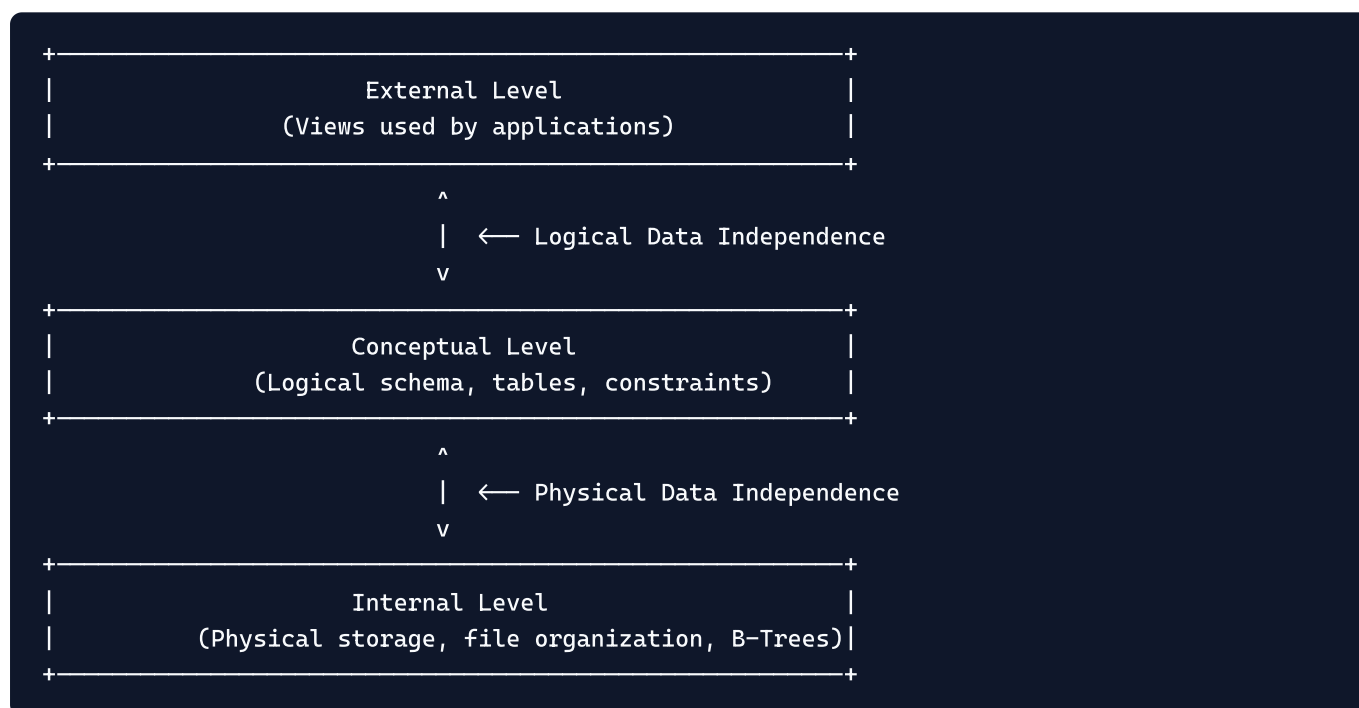
Explanation: Phantom reads occur when a concurrent transaction inserts new records matching a search filter between two read queries within the same transaction.

Q10. What does the ON DELETE CASCADE referential action do?

- A) Prevents deletion of the parent row
- B) Automatically deletes matching child rows when the referenced parent row is deleted
- C) Sets foreign keys in child rows to NULL
- D) Drops the entire table

Answer: B

Explanation: ON DELETE CASCADE automatically removes dependent rows in child tables whenever the corresponding master row is deleted.

 SECTION 2: High-Frequency VTU Short Q&A (4–6 Marks)
Q11. Compare Physical Data Independence with Logical Data Independence.**Answer:**

| Dimension | Logical Data Independence | Physical Data Independence | |---|---|---| | **Definition** | Ability to modify the conceptual schema without altering external schemas (application views). | Ability to modify internal storage structures without altering the conceptual schema. | | **Example** | Adding a new table, adding an attribute, or merging relations without breaking user apps. | Converting an index from B-Tree to Hash index, moving files to SSD without changing SQL queries. | | **Difficulty** | Much harder to achieve because applications depend heavily on logical data structure. | Easier to achieve as physical storage details are completely abstracted by the DBMS engine. |

Q12. Explain the ACID properties of database transactions with real-world examples.

Answer: A transaction is a logical unit of database processing. The **ACID** properties are:

1. Atomicity (All-or-Nothing):

- *Example:* In a bank transfer of ₹5,000 from Account A to Account B, if debited from A but the system crashes before crediting B, the transaction rolls back so A does not lose money.

2. Consistency (Integrity Preserving):

- *Example:* The total sum of money in Account A and Account B before the transfer must equal the sum after the transfer. Database constraints (e.g., balance ≥ 0) must remain valid.

3. Isolation (Independence of Concurrent Transactions):

- *Example:* If two users transfer money from Account A at the exact same millisecond, the DBMS serializes them so neither sees intermediate uncommitted balances.

4. Durability (Permanence):

- *Example:* Once a transaction commits and sends confirmation, the updates persist in non-volatile storage even if power cuts immediately after.

**SECTION 3: Detailed VTU Model Exam Questions (10–12 Marks)****Q13. [VTU Model QP - Module 4: Normalization Problem]**

Given a relational schema $R(A, B, C, D, E, F)$ with the following set of Functional Dependencies (FDs):

$F = \{ A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A \}$

(a) Find all Candidate Keys of relation R .

(b) Determine the highest Normal Form of R .

(c) Decompose R into BCNF with lossless join guarantee.

Answer:

Step 1: Attribute Closure to Find Candidate Keys

To find candidate keys, compute the closure (X^+) of attributes. Notice that attribute F does **not appear on the right-hand side** of any functional dependency! Therefore, F must be present in every candidate key.

- Let's test $(AF)^+$:
 - $(AF)^+ = \{A, F\}$
 - Using $A \rightarrow BC$: $\{A, B, C, F\}$
 - Using $B \rightarrow D$: $\{A, B, C, D, F\}$
 - Using $CD \rightarrow E$: $\{A, B, C, D, E, F\} = R$
 - Since $(AF)^+$ generates all attributes and no subset does, **AF is a Candidate Key.**
- Let's test $(BF)^+$:
 - $(BF)^+ = \{B, F\}$
 - Using $B \rightarrow D$: $\{B, D, F\}$
 - (Cannot determine A, C, E). Not a key.
- Let's test $(EF)^+$ (since $E \rightarrow A$):

- $(EF)^+ = \{E, F\}$
- Using $E \rightarrow A$: $\{A, E, F\}$
- Using $A \rightarrow BC$: $\{A, B, C, E, F\}$
- Using $B \rightarrow D$: $\{A, B, C, D, E, F\} = R$
- Therefore, **EF is also a Candidate Key.**
- Let's test $(CDF)^+$ (since $CD \rightarrow E$):
 - $(CDF)^+ = \{C, D, F\}$
 - Using $CD \rightarrow E$: $\{C, D, E, F\}$
 - Using $E \rightarrow A$: $\{A, C, D, E, F\}$
 - Using $A \rightarrow BC$: $\{A, B, C, D, E, F\} = R$
 - Since $B \rightarrow D$, can $(CBF)^+$ derive D ?
 - $(BCF)^+ = \{B, C, F\} \rightarrow \{B, C, D, F\} \rightarrow \{B, C, D, E, F\} \rightarrow R$.
 - Therefore, **BCF and CDF are also Candidate Keys.**

All Candidate Keys: $\{AF, EF, BCF, CDF\}$.

Prime Attributes: $\{A, B, C, D, E, F\}$ (All attributes are prime!).

Step 2: Determine Highest Normal Form

- **1NF:** Satisfied (all attributes atomic).
- **2NF:** A relation is in 2NF if no non-prime attribute is partially dependent on any candidate key. Since **all attributes are prime attributes**, there are NO non-prime attributes! Therefore, the relation is **trivially in 2NF**.
- **3NF Check:** For every $X \rightarrow Y$, either X is superkey or Y is prime.
 - In $A \rightarrow BC$: A is not superkey (needs F), but B and C are prime attributes. $\rightarrow$ Satisfies 3NF!
 - In $B \rightarrow D$: B is not superkey, but D is prime. $\rightarrow$ Satisfies 3NF!
 - In $CD \rightarrow E$: CD is not superkey, but E is prime. $\rightarrow$ Satisfies 3NF!
 - In $E \rightarrow A$: E is not superkey, but A is prime. $\rightarrow$ Satisfies 3NF!
 - **Highest Normal Form is 3NF!**
- **BCNF Check:** In BCNF, for every $X \rightarrow Y$, X MUST be a superkey.
 - In $B \rightarrow D$, B is NOT a superkey. Hence, R is **NOT in BCNF**.

Step 3: Decompose into BCNF

Violating dependency: $B \rightarrow D$ (where $B^+ = \{B, D\}$).

1. Decompose into:

- $R_1(B, D)$ with key B . (In BCNF!)
- $R_2(A, B, C, E, F) = R - \{D\}$.

2. Check R_2 :

- In R_2 , candidate keys are AF and EF .

- Check $A \rightarrow BC$: A is not superkey. $A^+ = \{A, B, C\}$.
 - Decompose R_2 into:
 - $R_{21}(A, B, C)$ with key A . (In BCNF!)
 - $R_{22}(A, E, F)$ with key EF (and AF). Since $E \rightarrow A$, decompose into $R_{221}(E, A)$ and $R_{222}(E, F)$. **Final BCNF Relations:**
 $R_1(B, D)$; $R_2(A, B, C)$; $R_3(E, A)$; $R_4(E, F)$.
- This decomposition is **lossless** because common attributes form candidate keys in the respective decomposed relations.

Q14. [VTU Model QP - Module 3: SQL Master Problem]

Consider the following relational schema for a Company Database:

- EMPLOYEE (EmpId, EmpName, Salary, DeptNo, ManagerId)
- DEPARTMENT (DeptNo, DeptName, Location)
- PROJECT (ProjNo, ProjName, DeptNo, Budget)
- WORKS_ON (EmpId, ProjNo, Hours)

Write SQL queries for the following:

1. Retrieve names of employees who earn more than the average salary of their department.
2. Find the department name that manages the maximum number of projects.
3. List the employees who work on ALL projects located in 'Bengaluru'.
4. Create a trigger that prevents inserting an employee with a salary less than ₹20,000.

Answer:

1. Correlated Subquery for Salary Greater than Department Average:

```
SELECT e1.EmpName, e1.Salary, e1.DeptNo
FROM EMPLOYEE e1
WHERE e1.Salary > (
    SELECT AVG(e2.Salary)
    FROM EMPLOYEE e2
    WHERE e2.DeptNo = e1.DeptNo
);
```

2. Department Managing Maximum Projects:

```

SELECT d.DeptName, COUNT(p.ProjNo) AS ProjectCount
FROM DEPARTMENT d
JOIN PROJECT p ON d.DeptNo = p.DeptNo
GROUP BY d.DeptNo, d.DeptName
HAVING COUNT(p.ProjNo) ≥ ALL (
    SELECT COUNT(ProjNo)
    FROM PROJECT
    GROUP BY DeptNo
);

```

3. Employees Working on ALL Projects in 'Bengaluru' (Relational Division using **NOT EXISTS**):

```

SELECT e.EmpName
FROM EMPLOYEE e
WHERE NOT EXISTS (
    -- Set of all projects in Bengaluru
    SELECT p.ProjNo
    FROM PROJECT p
    JOIN DEPARTMENT d ON p.DeptNo = d.DeptNo
    WHERE d.Location = 'Bengaluru'
    EXCEPT
    -- Set of projects this employee works on
    SELECT w.ProjNo
    FROM WORKS_ON w
    WHERE w.EmpId = e.EmpId
);

```

4. Trigger to Enforce Minimum Salary Constraint:

```

DELIMITER //
CREATE TRIGGER check_min_salary_before_insert
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
    IF NEW.Salary < 20000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Employee salary cannot be less than INR 20,000!';
    END IF;
END //
DELIMITER ;

```

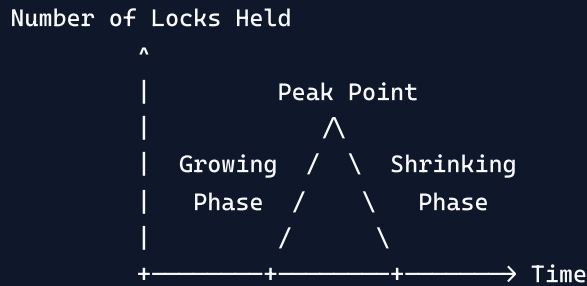
Q15. [VTU Model QP - Module 5: Concurrency Control & Recovery]

- (a) Explain Two-Phase Locking (2PL). Differentiate between Basic 2PL, Conservative 2PL, and Strict 2PL.
- (b) How does Write-Ahead Logging (WAL) and Checkpointing guarantee durability during crash recovery?

Answer:

Part (a): Two-Phase Locking (2PL)

2PL is a concurrency control protocol that guarantees **Conflict Serializability**.



1. **Growing Phase:** Transaction may acquire new locks, but cannot release any lock.
2. **Shrinking Phase:** Transaction may release locks, but cannot acquire any new lock.

Type of 2PL	Lock Acquisition	Lock Release	Deadlock Possible?	Cascading Abort Possible?
Basic 2PL	Acquired dynamically during growing phase	Released dynamically during shrinking phase	Yes	Yes
Conservative 2PL	Must acquire all required locks before execution starts	Released during shrinking phase	No (Deadlock-Free)	Yes
Strict 2PL	Acquired dynamically	Exclusive (X) locks held until commit/abort	Yes	No (Recoverable)
Rigorous 2PL	Acquired dynamically	ALL locks (Shared & Exclusive) held until commit	Yes	No (Strict Schedule)

Part (b): WAL and Checkpointing

1. Write-Ahead Logging (WAL):

- No data item is written to the physical database disk until the corresponding log record describing the update (including `Transaction_ID`, `Data_Item`, `Old_Value`, `New_Value`) is flushed to non-volatile log storage.
- Ensures that in case of sudden power crash, the DBMS can **UNDO** uncommitted transactions and **REDO** committed ones.

2. Checkpoints:

- Periodically, the DBMS flushes all dirty memory buffers to disk and writes a `<CHECKPOINT L>` record to the log.
- **Recovery Benefit:** During crash recovery, the DBMS only scans the log back to the most recent checkpoint rather than scanning from the beginning of database history, reducing recovery time from hours to seconds.

💡 Key Takeaway Checklist for Exam Day

- [] Practice 3NF and BCNF decomposition step-by-step; attribute closure calculation is guaranteed marks.
- [] Memorize SQL join syntax (`LEFT JOIN`, `INNER JOIN`, correlated subqueries).
- [] Understand difference between Conflict Serializability (tested via Precedence/Wait-for Graph) and View Serializability.
- [] Draw clear ER notations: entity (rectangle), attribute (oval), relationship (diamond), primary key (underlined attribute).